TP CRYPTO

Vous allez dans un premier temps implémenter un AES 128 :

Classe AES

constantes :

taille_bloc = 128
Rcon : int[] (voir cours)
sBox : int[][] (voir cours)
sBoxInv : int[][] (voir cours)

attributs :

taille_cle_maitre = 128 (puis 192 ou 256)
nb_ronde = 10 (puis 12 ou 14 en fonction de la taille de la clé maître)
master_Key = tableau multidimensionel d'entiers
keys = tableau multidimensionel d'entiers

Réfléchissez à de quelle manière vous allez stoker un octet

méthodes :

AES(taille-cle-maitre) : le constructeur , initialise les attributs (à faire petit à petit ...)

stringToBits(String message) : int[] transforme une chaîne de caractères en un tableau d'entiers : 0 et 1

bitsToString(int[] blocs) : String : transforme un tableau d'entiers (0 ou 1) en chaîne de caractères.

genereMasterKey() : génère une clé maître aléatoire de 128 bits. (puis 192 puis 256)

createKeys() : génère les clés de rondes.

b**ourrage**(int[]) : int[]

debourrage(int[]) : int[]

decoupage_en_blocs(int[]) : int[]...[] découpage du message binaire en blocs de 128 bits

recollage_des_bloc(int[]...[] blocs) : int[] recolle tous les blocs …

xor (int[] tab1, int[] tab2) : int[] réalise le xor entre tab1 et tab2

addRoundKey(…) … cf poly AES

subByte(...) : … cf poly AES

subByteInv () : ... cf poly AES

shiftRows(…) : … cf poly AES

shiftRowsInv(…) : ... cf poly AES

deux_a(…) : …

trois_a(…) : …

neuf_a(…) : …

onze_a(…) : …

treize_a(…) : …

quatorze_a(…) : …

prodAu(…) : …

prodAuInv(…) : …

mixColumns(…) : …

crypte(message : String) : int[]

decrypte(messCode : int[]) : String

remarque : au final les seuls membres publics de la classe seront le constructeur et les méthodes crypte et decrypte, tout le reste sera privé.

N'oubliez pas les commentaires dans votre code et faites une Javadoc.

Faire une classe TestAES pour tester votre classe AES.

deuxième version : faire AES avec des clés maître de taille 192 et 256 bits.

Quatrième version : interface graphique ….

Indications :

1. Faites une classe TestAES dans laquelle vous mettrez le main testant au fur et à mesure vos méthodes.

2. Pour les méthodes int[] **stringToBits**(String message) et **String bitsToString**(int[] blocs)
il faudra vous intéresser aux classes :

String
Integer

Byte

vous pourrez ainsi utiliser tous les caractères de l'alphabet en minuscules et majuscules ainsi que l'espace pour séparer les mots.

Si vous voulez utiliser tous les caractères possibles (avec les accents, ponctuation etc.…) il faudra chercher encore davantage.

3. Ordre dans lequel vous pouvez implémenter et tester les méthodes de la classe DES :

1) AES

2) stringToBits

3) bitsToString

4) genereMasterKey

5) bourrage

6) debourrage

7) decoupage_en_blocs

8) recollage_des_blocs

9) xor

10) subByte

11) create_Keys

12) addRoundKey

13) subByte

14) subByteInv

15) shiftRows

16) shiftRowsInv

17) deux_a

18) trois_a

19) neuf_a

20) onze_a

21) treize_a

22) quatorze_a

23) prodAu

24) prodAuInv

25) mixColumns

26) crypte(message : String) : int[]

27) decrypte(messCode : int[]) : String